Tsunagu: GraphRAG versus vector RAG on cross-referenced technical documentation

Fayçal Raghibi

22 October 2026

Abstract

Tsunagu (*to connect*) is a one-day personal prototype that asks whether a graph layer (explicit page links, code symbols and relations extracted by a language model) improves answers to relational, multi-hop and enumerative questions over a heavily cross-referenced technical documentation, compared with a dense vector RAG and a cheap hybrid RAG (BM25 and dense retrieval fused with reciprocal rank fusion), and at what cost. The corpus is the LangGraph documentation pinned at one commit. All three systems share the same chunks, the same answering model, the same prompt and the same top-k budget, so the retriever is the only variable. This report describes the project structure, every component, the experimental protocol and the results. Result numbers are not typed by hand: they are generated by `tsunagu report` and included verbatim.

Contents

1 Status of this document **3**

2 Research question **3**

3 Provenance and honesty contract **3**

4 Project structure **4**
4.1 Pipeline 4
4.2 Repository layout 5
4.3 Engineering conventions 5

5 Corpus and ingestion **5**
5.1 Source 5
5.2 Cleaning MDX 6
5.3 Chunking 6

6 The three systems **6**
6.1 A. Dense 6
6.2 B. Hybrid (BM25 + dense, RRF) 6
6.3 C. Graph 7
6.3.1 Graph construction 7
6.3.2 Entity resolution 7
6.3.3 Graph retriever 7

7 Answering **8**

8 Evaluation protocol **8**
8.1 Question set 8
8.2 Retrieval metrics (page level, $k = 8$) 9
8.3 Answer quality 9
8.4 Judge calibration 10
8.5 Cost 10

9 Results **10**

10 Findings **12**

11 Limits **12**

12 What I would do at Safran **12**

13 Reproducing **12**

A Prompts **12**

B Configuration **13**

1 Status of this document

- **Done:** corpus, ingestion, dense and hybrid indexes, graph construction and entity resolution, merge review, retrieval-only evaluation, full evaluation (30 questions, 3 systems, 3 repeats, 270 answered and judged rows, no parse failure).
- **Pending:** judge calibration (human gate G3). Twenty blind items in `data/labels_manual.csv` wait for the owner's scores. Until they exist, every correctness number in Section 9 comes from an LLM judge whose agreement with a human is *not measured yet*. The calibration line in the results says so.
- **Owner sections:** Findings, Limits and "What I would do at Safran" (Sections 10 to 12) are written by the owner only.

To refresh the numbers after calibration: run `tsunagu calib`, then `tsunagu report`, then rebuild this PDF.

2 Research question

Question. On a corpus of technical documentation full of cross-references, does a graph layer (entities, relations, explicit links between pages) improve answers to relational, multi-hop and enumerative questions, compared with (1) a dense vector RAG and (2) a hybrid RAG (BM25 plus dense, fused with reciprocal rank fusion)? And what does it cost in indexing tokens, indexing time, query latency and entity-resolution effort?

Why a hybrid control. Many published "GraphRAG wins" are really "anything beats naive dense retrieval" wins. A lexical-plus-dense hybrid is cheap and strong on exact identifiers. Comparing the graph with the hybrid, not only with dense, separates the value of the *graph* from the value of *better retrieval in general*.

Why this corpus. The LangGraph documentation has the properties of industrial technical documentation, which motivates the comparison:

LangGraph docs	Industrial technical documentation
Links between pages	References between documents
API symbols (`StateGraph`, `InMemorySaver`)	Exact references: part numbers, document ids
Deprecated APIs, version notes	Revisions, applicability by version
"Feature X requires Y"	"Procedure X applies under condition Y"

3 Provenance and honesty contract

The project follows a written honesty contract (`docs/graphrag_prototype_spec.md`, Section 0). Its rules, and the places where reality differed from the plan:

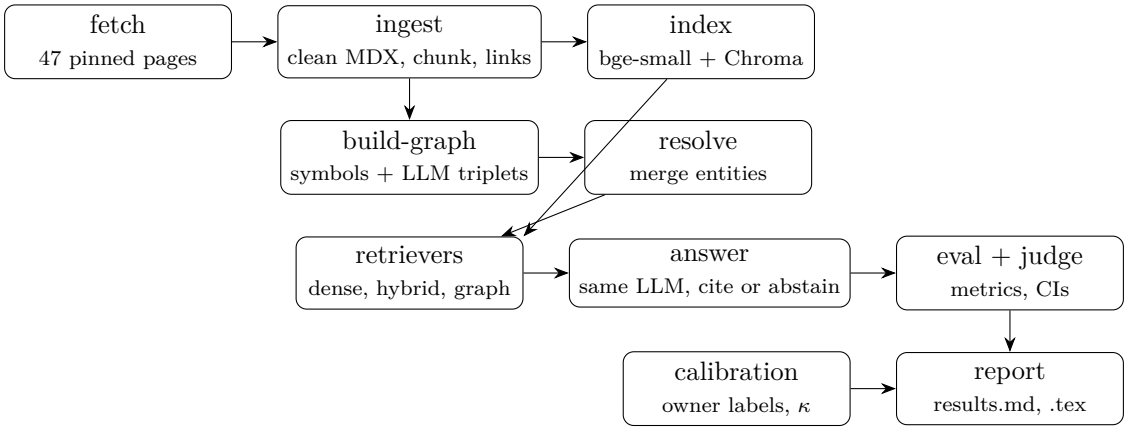
1. **Numbers.** Every result number comes from `results/results.md`, generated by `tsunagu report`. This document includes the same numbers through `results/results_auto.tex` and `results/observations.tex`, which the same command writes by converting the same Markdown. Nothing is transcribed by hand.
2. **Question set and merge labels.** The plan had the owner writing the 30 questions and labelling the merge review. On 22 October 2026 the owner asked the AI assistant to do both. They were written from the cleaned corpus *before any retrieval or answer result existed*, and the question file was hashed (`results/questions_lock.json`) before the first run. Consequences, recorded in the design addendum (fact 5): the question set may favour

structures the graph handles well, because the same assistant designed the graph retriever; the false-merge rate reflects the assistant's judgement, not a human one.

3. **Judge calibration stays human.** `data/labels_manual.csv` is owner-only: a model labelling answers to calibrate a model judge would defeat its purpose.
4. **One revision after seeing results.** The graph retriever was revised once after the first retrieval-only run, because it had collapsed into dense retrieval (Section 6.3.3). This is a stated limit.
5. **Conclusions.** Findings, Limits and the Safran section are the owner's.

4 Project structure

4.1 Pipeline



Each box is a `tsunagu` command (Typer CLI). Every LLM call goes through one cached client, so re-running any stage costs nothing once its calls are cached.

4.2 Repository layout

Path	Role
`config.yaml`	Single source of settings: page list, LLM, chunking, retrieval, graph, evaluation.
`aliases.yaml`	Hand-written alias map for entity resolution (canonical name: aliases).
`src/tsunagu/config.py`	Pydantic models for `config.yaml`.
`corpus.py`	Downloads the 47 pages at the pinned commit.
`mdx.py`	Turns Mintlify MDX into plain Markdown, Python variant only.
`links.py`	Normalises internal links to page ids.
`chunking.py`	Heading-aware chunker that never splits a code block; symbol extraction.
`ingest.py`	Writes chunks, link edges, corpus statistics and a page index.
`tokens.py`	cl100k token counting.
`llm.py`	OpenAI-compatible client for NVIDIA NIM: disk cache, rate limit, retries, usage log.
`embeddings.py`, `store.py`	bge-small embedder; in-memory chunk store.
`vector_index.py`, `index.py`	ChromaDB dense index and saved embedding matrices.
`retrievers.py`	Dense, BM25 and hybrid (RRF) retrievers.
`extract.py`	Schema-guided triplet extraction (LlamaIndex backend and JSON backend).
`graph.py`	JSON property graph: nodes, edges, provenance, statistics.
`resolve.py`	Entity resolution, merge log, random merge review sample.
`graph_retriever.py`	Graph retriever: seeds, expansion, ranking.
`systems.py`	Builds the three systems from config.
`answer.py`	Shared answering prompt, JSON parsing, one retry, abstention.
`judge.py`	Correctness, faithfulness and citation-precision judges.
`questions.py`	Question model and validator.
`metrics.py`	hit@k, recall@k, MRR, Wilson interval, Cohen's κ, percentiles.
`runner.py`	Runs retrieval, answering and judging for every question and system.
`calibration.py`	Blind label sample and judge-versus-human agreement.
`report.py`	Generates the tables, observations and their LaTeX copies.
`cli.py`	Typer commands.
`tests/`	22 test modules, 102 tests, run offline with fake LLMs and embedders.
`data/`	Raw corpus, processed chunks, graph files, questions, manual labels.
`index/`	Chroma database and embedding matrices.
`results/`	Every generated artefact: rows, logs, merge review, calibration, report.
`cache/llm/`	One file per LLM call, keyed by its full request.
`docs/`	Spec, design addendum, implementation plan, this report.

4.3 Engineering conventions

Python 3.12 in a dedicated virtual environment; type hints and `from __future__ import annotations` everywhere; Google docstrings. Each behaviour was written test first. `ruff check`, `ruff format`, `mypy` and `pytest` run before every commit through pre-commit, and again in GitHub Actions. Work happens on the `feat/mvp` branch with Conventional Commits; merging to `main` needs the owner's explicit approval.

5 Corpus and ingestion

5.1 Source

The LangGraph documentation moved from `langchain-ai/langgraph` to `langchain-ai/docs` and is now written in Mintlify MDX. The corpus is pinned at commit

`23152b1133e7759381661e93bcdca778101a80bc`.

It contains all of `src/oss/langgraph/` except two changelog stubs and the case studies, plus the concept pages on memory, human-in-the-loop and context, and the LangChain pages on human-in-the-loop and multi-agent systems (multi-agent moved to the LangChain section). That gives 47 pages; the exact list is in `config.yaml`. Size and chunk counts appear in the first line of the results.

5.2 Cleaning MDX

`mdx.py` turns each page into plain Markdown:

- front matter is dropped (the title is kept), as are `{/* */}` comments, snippet imports and lines that are only JSX tags (`<Note>`, `<Tabs>`, `<Accordion>`);
- `:::python` blocks are unwrapped and `:::js` blocks removed, so the corpus is not duplicated in TypeScript;
- API references written `` @[`StateGraph`] `` become code spans. They are a free and precise symbol signal;
- code fences are copied unchanged.

Internal links are absolute (`/oss/langgraph/persistence`, sometimes `/oss/python/langgraph/...`); `links.py` normalises them to page ids such as `langgraph/persistence`. Links that leave the corpus are counted but not kept as edges.

5.3 Chunking

Pages are split on headings outside code. Inside a section, paragraphs and whole code blocks are packed greedily up to 500 tokens (cl100k), with about 50 tokens of trailing prose carried over as overlap; a block is only split if it is larger than 900 tokens, and code blocks are never cut mid-block unless they exceed that limit. Each chunk keeps its heading path; its citation label is `page#last heading`. Symbols are collected from prose code spans and from `from x import A, B` lines in code.

6 The three systems

All three retrieve $k = 8$ chunks from the same 732-chunk store and pass them to the same answering prompt.

6.1 A. Dense

Chunks are embedded with `BAAI/bge-small-en-v1.5` (384 dimensions, normalised) and stored in ChromaDB. A query is embedded with the model's query instruction and the 8 chunks with the highest cosine similarity are returned.

6.2 B. Hybrid (BM25 + dense, RRF)

BM25 runs over a tokeniser that lowercases words and also splits camelCase and snake_case identifiers, so `InMemorySaver` matches "in memory saver". Each retriever returns 20 candidates, fused with reciprocal rank fusion:

$$\mathrm{RRF}(d) = \sum_{r \in \{\mathrm{bm25},\, \mathrm{dense}\}} \frac{1}{k_{\mathrm{rrf}} + \mathrm{rank}_r(d)}, \qquad k_{\mathrm{rrf}} = 60,$$

with ranks starting at 1. The top 8 fused chunks are returned.

6.3 C. Graph

6.3.1 Graph construction

The graph is a JSON property graph (`data/graph/*.json`), chosen over a library store because one file can be read, diffed and traversed by hand.

Deterministic layer (free). One `Page` node per page; `LINKS_TO` edges from the normalised links; one `Symbol` node per code symbol, with `Page -MENTIONS-> Symbol` edges. Every node stores the ids of the chunks where it appears (`chunk_ids`), which is how traversal gets back to text.

LLM layer. Each chunk is sent to the extraction model with a fixed schema:

- entity types `Concept`, `Class`, `Function`, `Parameter`, `Feature`;
- relation types `USES`, `REQUIRES`, `CONFIGURES`, `PART_OF`, `ALTERNATIVE_TO`, `DEPRECATED_BY`, `RETURNS`;
- at most 10 triplets per chunk; names copied as written.

A strict filter drops anything outside the schema, empty names and self-loops.

Extraction backend: a measured choice. The plan was LlamaIndex `SchemaLLMPathExtractor`, with our own JSON prompt as fallback, decided by a 10-chunk spike. On the real model, LlamaIndex returned 0 triplets on all 10 chunks *without raising any error*, while the JSON extractor returned triplets on 9 of 10 (`results/extraction_spike.json`). The JSON backend was used for the whole graph. The lesson: a framework that swallows extraction errors needs a measured check before it is trusted.

6.3.2 Entity resolution

Duplicates are merged with a union-find, in four steps; `Page` nodes are never touched:

1. **normalize**: same type and same normalised name (lowercase, no backticks or `()`, separators collapsed, last word singularised);
2. **alias**: same canonical name after the hand-written `aliases.yaml`;
3. **symbol_link**: a deterministic `Symbol` joins the LLM code entity (`Class`, `Function`, `Parameter`) with the same name. This is the bridge between the free skeleton and the LLM layer;
4. **embedding**: within a type, entity names whose bge-small cosine similarity is above 0.92 are merged, most similar pairs first.

Every union is logged in `results/merges.csv`. To estimate the false-merge rate without bias, 30 embedding merges are drawn at random (seed 42) into `results/merges_review.csv` and labelled correct or false. Counts per step and the false-merge count are in the "Graph" line of the results.

6.3.3 Graph retriever

1. **Seeds.** Query terms are code spans plus word 1- to 3-grams (stopwords and words under 3 characters dropped), normalised like entity names. Any entity whose name or alias matches is a seed. In addition, the 5 entities whose name embedding is closest to the query become seeds if their similarity is at least 0.6.
2. **Expansion.** Breadth-first, 2 hops, over all relations (`LINKS_TO`, `MENTIONS` and the LLM relations). From each node at most 40 neighbours are followed, chosen by name similarity to the query.

3. **Pool.** Every chunk recorded on a reached node joins the pool, tagged with the smallest hop at which it was reached.
4. **Ranking.** Pool chunks are ranked by $\text{score}(c) = \cos(q, c) - \lambda \cdot \text{hop}(c)$, ties broken by chunk id. If the pool holds fewer than 8 chunks, it is topped up with the best dense chunks, and the number of such fallback chunks is recorded.

`tsunagu ask "..." -system graph -show-path` prints the seeds, the edges followed and the hop of each returned chunk.

Version 1 collapsed into dense. In the first retrieval-only run, the 2-hop expansion reached 630 to 732 of the 732 chunks, so ranking the pool by similarity returned exactly dense's top 8 on 29 of 30 questions (rows kept in `results/retrieval_rows_graph_v1.jsonl`). Measured causes: generic one-word seeds ("default", "graph", "value") and hub entities (`LangGraph` in 180 chunks, `StateGraph` in 138, `InMemorySaver` in 112) that connect most of the corpus.

Revision 2. Three rules, with values set from corpus statistics before re-running, not tuned against the questions:

(a) an entity mentioned in more than 2% of chunks (14 chunks) is a *hub* and is neither a seed nor traversed (60 hubs; pages are exempt because they route `LINKS_TO`);

(b) hop penalty $\lambda = 0.05$ per hop from the seeds;

(c) a plain lowercase one-word match only seeds code entities (`Symbol`, `Class`, `Function`, `Parameter`), unless the word is also in a code span.

After revision 2, the graph's top 8 equals dense's on 1 of 30 questions. The pool stays large because pages link widely (the mean pool size is in the latency table). Setting the three values to `null`, `0.0` and `false` in `config.yaml` reproduces version 1. The revision is covered by six unit tests (hop penalty, tie-break, hubs never seeded or traversed, pages never hubs, plain-word filter, filter off by default).

7 Answering

One prompt for all systems (`answer.py`, reproduced in Appendix A). The model must use only the context, cite the label of every supporting passage, and reply with one JSON object `{answer, citations, abstained}`. If the context does not contain the answer, it must answer `INSUFFICIENT_CONTEXT` and set `abstained` to true. An invalid JSON reply gets exactly one retry with a correction message; failures are counted.

Model. NVIDIA NIM, `nvidia/nemotron-3.5-lightning-30b-a3b`, temperature 0, thinking disabled, at most 2048 output tokens, limited to 35 requests per minute. The same model extracts, answers and judges; this is a stated limit.

LLM client. Each call is cached on disk under a key made of the full request plus a tag (system, question, repeat), so repeats are real separate calls but re-runs are free. The client retries with backoff, logs latency and tokens per stage, and `tsunagu usage` sums them.

8 Evaluation protocol

8.1 Question set

30 questions with gold answers and gold pages, in five types:

Type	n	Example
single_fact	7	What is LangGraph's default recursion limit, and which exception is raised when a graph reaches it?
multi_hop	8	A node writes a record through an external API and then calls `interrupt()`. What happens to that API call when the graph resumes, and how should the node be designed?
relational	8	Which checkpointer implementations do the docs list, and which are described as suited for production?
exact_reference	4	Which error code covers a node that returns a list instead of a dict, and what exception message does it show?
unanswerable	3	How do I deploy a LangGraph agent on AWS Bedrock Agents?

Unanswerable questions name products absent from the corpus (checked by search). The set was validated by `tsunagu check-questions` and hashed before the first run.

8.2 Retrieval metrics (page level, $k = 8$)

Retrieved chunks are mapped to pages in order of first appearance. With G the gold pages and P_k the retrieved pages:

$$\text{hit@}k = \mathbf{1}[\, G \cap P_k \neq \emptyset \,], \qquad \text{recall@}k = \frac{|G \cap P_k|}{|G|}, \qquad \text{MRR} = \frac{1}{\text{rank of the first gold page}}.$$

They are computed on the 27 answerable questions.

8.3 Answer quality

Correctness. The judge compares the answer with the gold answer: 2 = correct and complete, 1 = partly correct, 0 = wrong, contradicting, irrelevant or refusing. Extra correct detail does not lower the score. An abstention on an unanswerable question scores 2 without calling the judge; an abstention on an answerable one scores 0.

Pass rule. Each (question, system) is answered and judged 3 times. A question *passes* for a system when it scores 2 in a majority of the repeats. Pass rates use n = number of questions, not 90 runs, which would overstate independence. *Repeat stability* is the share of questions whose 3 scores are identical.

Confidence intervals. 95% Wilson score intervals for a proportion $\hat{p}$ over n questions, $z = 1.96$:

$$\frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}.$$

With about 30 questions, differences under about 15 points are probably noise; per-type n is 3 to 8, so per-type rates are indicative only.

Faithfulness. The judge splits the answer into atomic claims and marks each as supported or not by the retrieved passages. Reported as supported claims over all claims, pooled over repeats.

Citation precision. A cited page is *precise* if it is a gold page or supports at least one supported claim. Reported as precise citations over all citations.

Abstention. Correct abstention: share of unanswerable runs that abstain. False abstention: share of answerable runs that abstain.

8.4 Judge calibration

`tsunagu calib-sample` draws 20 judged answers, balanced across systems, into `data/labels_manual.csv` with the system hidden; the judge's scores go into a separate key file the labeller does not open. The owner scores each answer 0/1/2 against the gold answer. `tsunagu calib` then reports raw agreement and unweighted Cohen's κ,

$$\kappa = \frac{p_o - p_e}{1 - p_e},$$

on the 0/1/2 scale and on the pass/fail view. Rule fixed in advance: if $\kappa < 0.6$, the judge is not trusted and the results are reported as such.

8.5 Cost

Indexing cost: extraction calls, tokens and LLM time, and the time to build the dense index. Query cost: retrieval latency (p50, p95), end-to-end latency, answer tokens per query, and for the graph the mean pool size and the number of dense fallback chunks.

9 Results

The text below is generated by `tsunagu report` (file `results/results_auto.tex`). It is identical in content to the AUTO block of `results/results.md`.

Read with care. Judge calibration is pending (see the calibration line below). Correctness, faithfulness and citation numbers come from an LLM judge whose agreement with a human has not been measured yet.

How to read the tables

- **Header lines.** Corpus size; graph size before and after resolution, merges per step and the false-merge count from the 30-row review; indexing cost.
- **Retrieval.** Three numbers per cell: hit@8 / recall@8 / MRR, per question type and over all answerable questions. Unanswerable questions have no gold page and are excluded.
- **Correctness.** Passing questions over questions, pass rate and 95% Wilson interval. "Mean score" averages the 0 to 2 scores over all runs. Overlapping intervals mean the difference is not distinguishable at this sample size.
- **Faithfulness, citations, abstention.** Pooled over the 3 repeats.
- **Latency and tokens.** Per query; graph-only rows are `n/a` for the other systems.

Corpus: LangGraph docs @ `23152b1133e7`, 47 pages, 178726 tokens (cl100k), 732 chunks

Graph: 4170 entities before / 3237 after resolution (3648 LLM entities before, 3100 after), 5097 LLM relations, 129 explicit links; merges {'normalize': 283, 'alias': 28, 'symbol_link': 356, 'embedding': 266}; false merges found: 13/30 reviewed

Indexing: 629244 LLM tokens over 751 extraction calls, 67.7 min of LLM time (extraction model: nvidia/nemotron-3.5-lightning-30b-a3b, backend: json); dense index 178.4 s (BAAI/bge-small-en-v1.5); chunks without triplets: 49/732

Retrieval (hit@8 / recall@8 / MRR, page level)

Type	dense	hybrid	graph
single_fact (n=7)	1.00 / 1.00 / 0.93	1.00 / 1.00 / 1.00	1.00 / 1.00 / 0.93
multi_hop (n=8)	1.00 / 0.79 / 0.94	1.00 / 0.85 / 1.00	1.00 / 0.85 / 0.85
relational (n=8)	1.00 / 0.88 / 0.94	1.00 / 0.88 / 1.00	1.00 / 0.94 / 0.94
exact_reference (n=4)	1.00 / 1.00 / 0.67	1.00 / 1.00 / 0.50	0.75 / 0.62 / 0.46
all (n=27)	1.00 / 0.90 / 0.90	1.00 / 0.92 / 0.93	0.96 / 0.88 / 0.84

Correctness (judge score 2 in a majority of repeats), 95% Wilson CI, n = questions

Type	dense	hybrid	graph
single_fact	4/7 = 57% [25%, 84%]	5/7 = 71% [36%, 92%]	5/7 = 71% [36%, 92%]
multi_hop	6/8 = 75% [41%, 93%]	6/8 = 75% [41%, 93%]	5/8 = 62% [31%, 86%]
relational	6/8 = 75% [41%, 93%]	4/8 = 50% [22%, 78%]	6/8 = 75% [41%, 93%]
exact_reference	4/4 = 100% [51%, 100%]	4/4 = 100% [51%, 100%]	3/4 = 75% [30%, 95%]
unanswerable	1/3 = 33% [6%, 79%]	1/3 = 33% [6%, 79%]	2/3 = 67% [21%, 94%]
answerable	20/27 = 74% [55%, 87%]	19/27 = 70% [52%, 84%]	19/27 = 70% [52%, 84%]
all	21/30 = 70% [52%, 83%]	20/30 = 67% [49%, 81%]	21/30 = 70% [52%, 83%]
mean score (0 to 2)	1.66	1.61	1.61
repeat stability	90%	87%	83%

Faithfulness, citation precision, abstention (pooled over repeats)

Metric	dense	hybrid	graph
faithfulness (supported claims)	662/662 = 100%	715/718 = 100%	607/609 = 100%
citation precision	119/131 = 91%	120/131 = 92%	116/127 = 91%
correct abstention (unanswerable)	44%	44%	56%
false abstention (answerable)	0%	0%	0%
JSON parse failures	0	0	0
judge unparseable	0	0	0

Latency and tokens per query

Metric	dense	hybrid	graph
retrieval p50 / p95 (ms)	51 / 173	62 / 90	60 / 254
end-to-end p50 / p95 (s)	2.1 / 8.3	2.4 / 6.5	2.2 / 6.8
answer tokens per query (in + out)	4910	5312	4822
graph candidate pool (mean chunks)	n/a	n/a	627
graph dense fallback (chunks, total)	n/a	n/a	0

Judge calibration: PENDING (data/labels_manual.csv not labelled yet).

Statistics: 30 questions, 3 repeat(s) per system. With about 30 questions, differences under about 15 points are probably noise; per-type n is 3 to 8, so per-type rates are indicative only.

Generated observations

`results/observations.tex`: for each question type, the two systems with the highest pass rate and whether their Wilson intervals overlap. These are facts, not conclusions.

- single_fact: highest pass rate hybrid 71%, next graph 71%; intervals overlap: not distinguishable at this n.
- multi_hop: highest pass rate hybrid 75%, next dense 75%; intervals overlap: not distinguishable at this n.
- relational: highest pass rate graph 75%, next dense 75%; intervals overlap: not distinguishable at this n.
- exact_reference: highest pass rate hybrid 100%, next dense 100%; intervals overlap: not distinguishable at this n.
- unanswerable: highest pass rate graph 67%, next hybrid 33%; intervals overlap: not distinguishable at this n.
- answerable: highest pass rate dense 74%, next hybrid 70%; intervals overlap: not distinguishable at this n.

10 Findings

[To be written by the owner: 3 to 5 honest bullets drawn from the tables above, including where the graph lost. Write after judge calibration.]

11 Limits

[To be written by the owner: corpus size, 30 questions, one LLM for extraction, answers and judge, judge bias, AI-drafted questions, graph retriever revised once after a retrieval run.]

12 What I would do at Safran

[To be written by the owner: 5 lines.]

13 Reproducing

```
./clean_setup.sh # or .\clean_setup.ps1 ; creates tsunagu-venv
cp .env.example .env # set NVIDIA_API_KEY
tsunagu fetch && tsunagu ingest && tsunagu check-questions
tsunagu index && tsunagu build-graph && tsunagu resolve && tsunagu merge-sample
tsunagu eval --retrieval-only # retrieval metrics only, no LLM calls
tsunagu eval # 3 systems x 3 repeats, answer + judge
tsunagu calib-sample # owner labels data/labels_manual.csv
tsunagu calib && tsunagu report # results.md, observations.md, .tex copies
cd docs/report && latexmk -pdf tsunagu.tex
```

Every LLM call is cached in `cache/llm/`, so a second run of any command makes no network call. Tests run offline: `./tsunagu-venv/Scripts/python -m pytest`.

A Prompts

Answering (system prompt)

```
You answer questions about the LangGraph documentation using ONLY the context passages provided.
Rules:
1. Use only facts stated in the context. Do not use outside knowledge.
2. Each passage starts with its label in square brackets, like [langgraph/persistence#
    Checkpointers]. Put the label of every passage that supports your answer in "citations",
    copied exactly.
3. If the context does not contain the answer, set "answer" to "INSUFFICIENT_CONTEXT", "citations"
     to [] and "abstained" to true.
4. Reply with one JSON object and nothing else:
{"answer": "<2 to 6 sentences>", "citations": ["<label>", "..."], "abstained": false}
```

Extraction

```
Extract a knowledge graph from this passage of the LangGraph documentation.
Entity types: Concept, Class, Function, Parameter, Feature.
Relation types: USES, REQUIRES, CONFIGURES, PART_OF, ALTERNATIVE_TO, DEPRECATED_BY, RETURNS.
Rules:
- Use names exactly as written in the passage; keep code names such as InMemorySaver unchanged.
- Only extract relations the passage states or directly shows.
- At most {max_triplets} triplets.
Reply with one JSON object only:
{"triplets": [{"subject": "...", "subject_type": "...", "relation": "...", "object": "...", "
    object_type": "..."}]}
```

```
Passage ({heading}):
{text}
```

Correctness judge

```
You grade an answer to a question about the LangGraph documentation.

Question: {question}
Reference answer: {gold}
Candidate answer: {answer}

Rubric:
2 = correct and complete: states every key point of the reference answer and contradicts none.
1 = partially correct: at least one key point is right, but a key point is missing or slightly
    wrong.
0 = wrong, contradicts the reference, irrelevant, or says it cannot answer.
Extra correct detail that is not in the reference does not lower the score.

Reply with one JSON object only: {"score": 0, "reason": "<one sentence>"} where score is 0, 1 or 2.
```

Faithfulness judge

```
Split the answer into atomic factual claims. For each claim, decide whether the context passages
    support it.

Context passages (each starts with its label in square brackets):
{context}

Answer:
{answer}

Reply with one JSON object only:
{"claims": [{"claim": "...", "supported": true, "support_labels": ["<label>"]}]}
```

B Configuration

`config.yaml` from line 53 (the page list above it is summarised in the corpus section).

```yaml
llm:
  provider: nvidia
  base_url: https://integrate.api.nvidia.com/v1
  model: nvidia/nemotron-3.5-lightning-30b-a3b
  api_key_env: NVIDIA_API_KEY
  temperature: 0.0
  max_tokens: 2048
  rpm: 35
  timeout_s: 120
  max_retries: 5
  disable_thinking: true

chunking:
  target_tokens: 500
  overlap_tokens: 50
  max_tokens: 900

retrieval:
  top_k: 8
```

```
  candidate_k: 20
  rrf_k: 60
  embed_model: BAAI/bge-small-en-v1.5

graph:
  extraction_backend: json # set by Task 16 spike (llamaindex dropped due to relation schema
      mismatch)
  max_triplets_per_chunk: 10
  merge_threshold: 0.92
  hops: 2
  seed_vector_k: 5
  seed_min_sim: 0.6
  max_expand_degree: 40
  # Revision 2 (2026-10-22, after the first retrieval run): v1 reached 630 to 732 of 732
  # chunks and returned dense's top 8 on 29/30 questions. Values chosen from corpus
  # statistics before re-running; see design doc fact 6. Null / 0.0 / false = v1.
  hub_chunk_frac: 0.02 # entity in more than 2% of chunks (14) = hub
  hop_penalty: 0.05 # similarity cost per hop from the seeds
  code_only_single_words: true # a plain one-word match only seeds code entities

evaluation:
  repeats: 3
  systems: [dense, hybrid, graph]
```